Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

División de Ciencias de la ingeniería

Ingeniería

Organización de Lenguajes y Compiladores 1 Sección “A”

Ing. Jose Moises Granados Guevara

Primer Semestre 2026

Estudiante:

Pablo Alejandro Maldonado de León

Carné: 202430233

Manual de usuario Práctica #2

Descripción:

En el siguiente manual se describe como hacer uso de la aplicación web. En esta ocasión **se presenta al lenguaje de analizadores sintácticos LL(1) Wison.** Este es un lenguaje creado para poder generar parsers partiendo de gramáticas NTPS. Donde este se compone de una parte léxica que permite registrar los tokens que se tomarán en cuenta en las producciones de la gramática. Y la parte sintáctica que es la estructura en la que se basará el LL(1) para dar por válidas las entradas.

“Bienvenido usuario”

Como primer punto se requieren ciertos recursos básicos para poder ejecutar el programa. Sin embargo no es de alta demanda ya que es una página web cargada en un servidor que mediante la conexión a internet se puede visitar y generar tus propios analizadores sintácticos online con la mejor experiencia de usuario por las tecnologias utilizadas:

Tecnologías:

- LALR Jison
- Svelte
- JavaScript
- REST API
- Node.js

Recomendaciones:

Debido a que es una aplicación web se debe considerar que es bastante portable la aplicación siempre y cuando se cuente con lo siguiente:

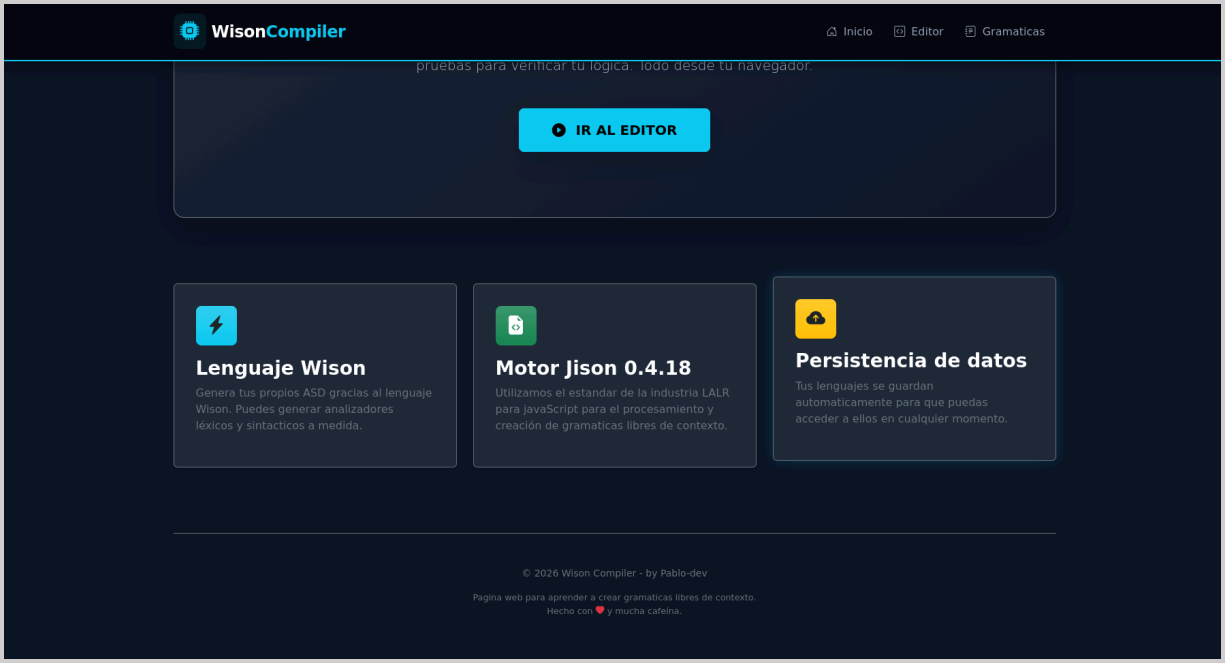
- Teléfono móvil o computadora.
- Conexión a internet
- **Opcional:** Tener bastante memoria RAM. Aunque no es obligatoria puede ayudar a agilizar más los procesos.

Pasos iniciales:

1. Encender el dispositivo
2. Ingresar a la URL **http://localhost:5173/gramaticas**
3. Iniciar la aplicación desde la landpage.
4. Empezar a crear tus gramaticas a tu gusto.

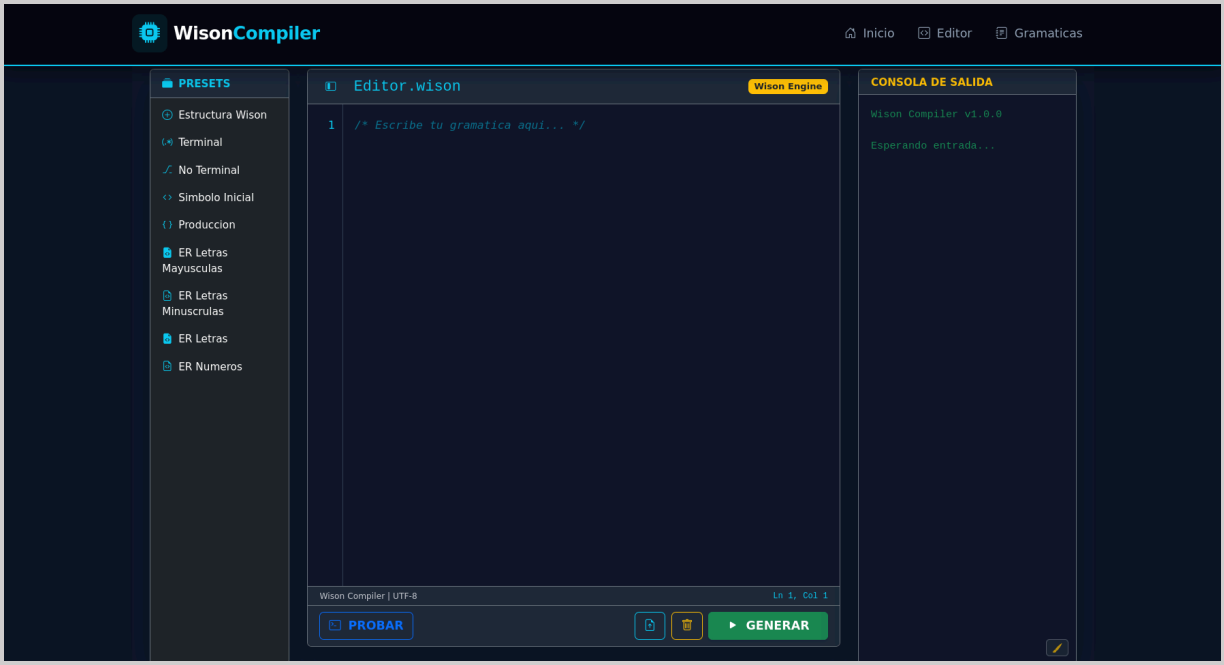
MENÚ PRINCIPAL

Esta es una presentación con la que cuenta la aplicación web ya que se le presentan todas las funcionalidades principales y el propósito de la aplicación. Esta es con fines de aprender a armar gramáticas NTPS de una forma más avanzada ya que es completamente personalizable:



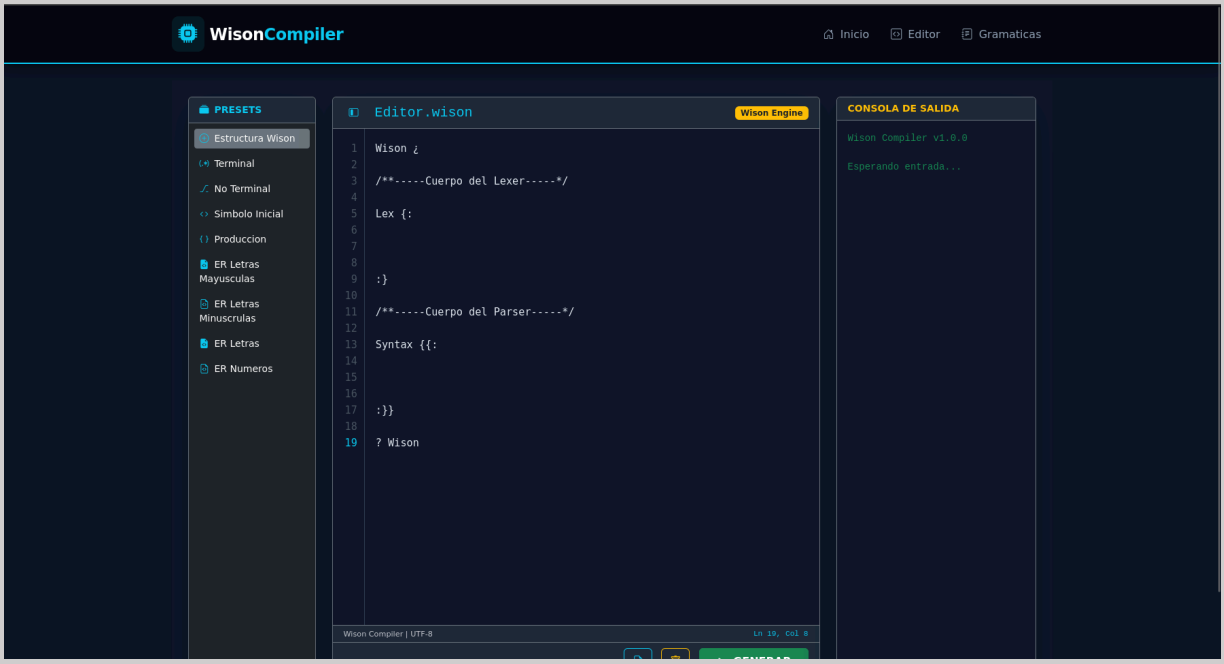
MENÚ DE EDITOR

Esta es la pantalla principal de la aplicación web ya que contiene el área de entrada donde el usuario directamente puede empezar a escribir en el lenguaje Wison siempre y cuando coloque la sintaxis correcta para poder armar su gramática NTPS.

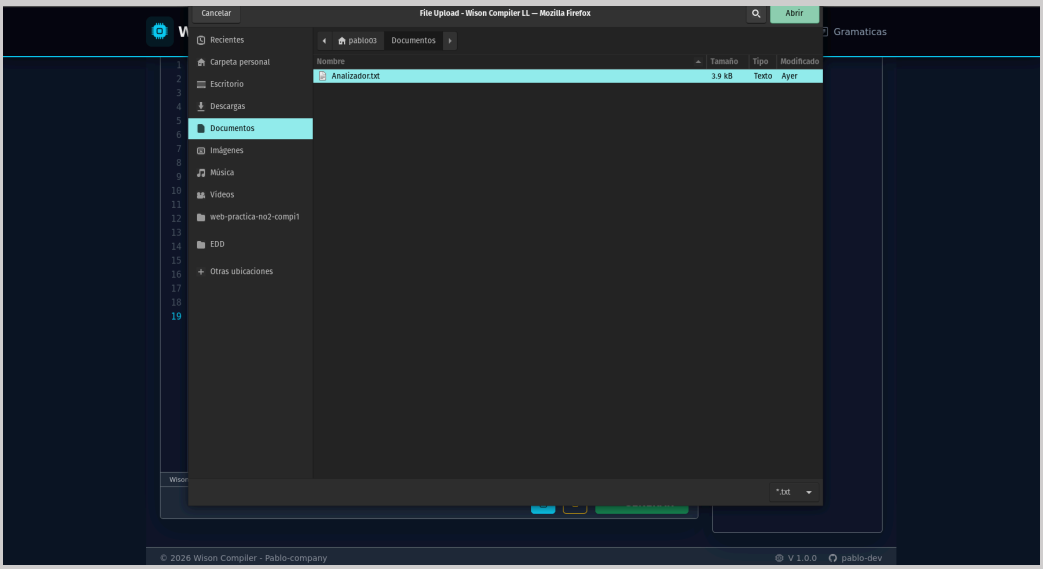


Opciones que ofrece la interfaz:

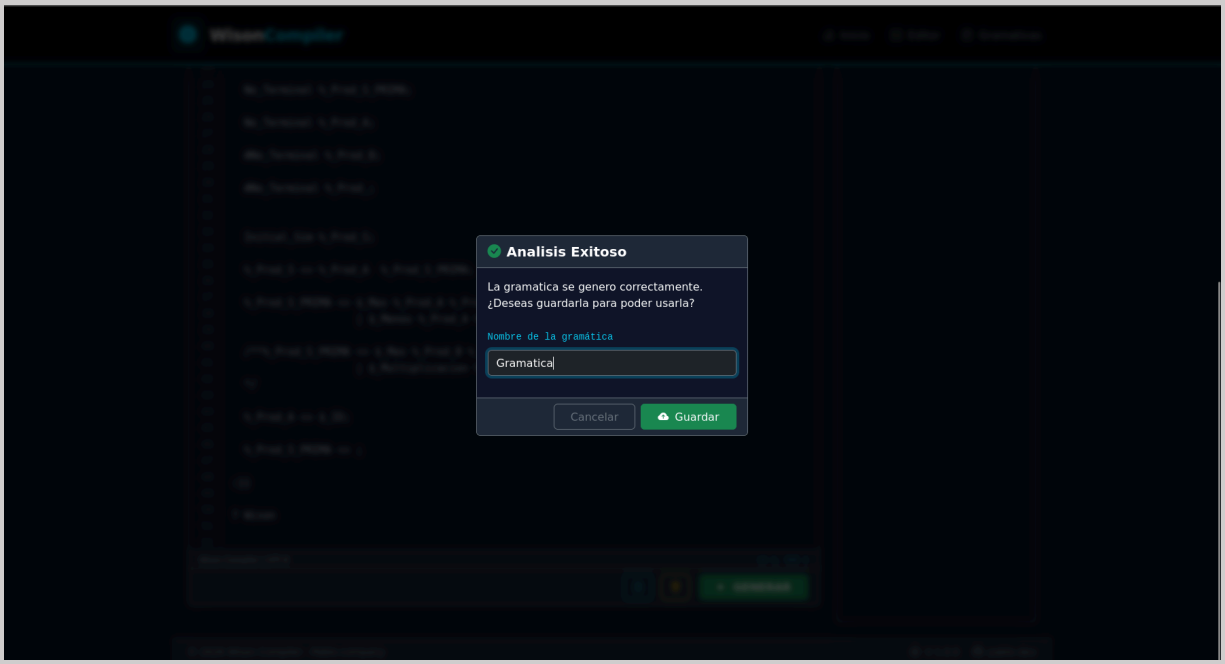
Programar con presets: Estos le permiten al usuario poder insertar modelos de código que pueden ser sustituidos posteriormente por lo que desee el usuario. Esto le permite al usuario una experiencia más amigable y por ende más fácil para poder generar sus gramáticas.

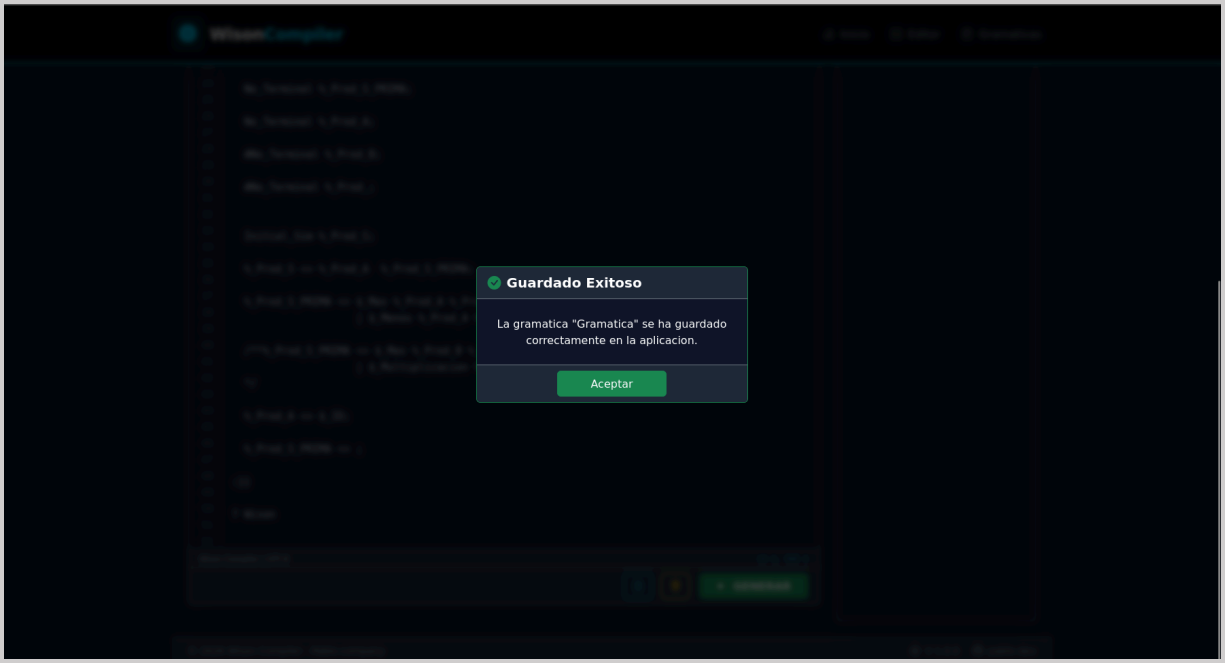


Cargar archivo: El usuario puede cargar un archivo de entrada para poder generar su gramática desde un archivo externo.

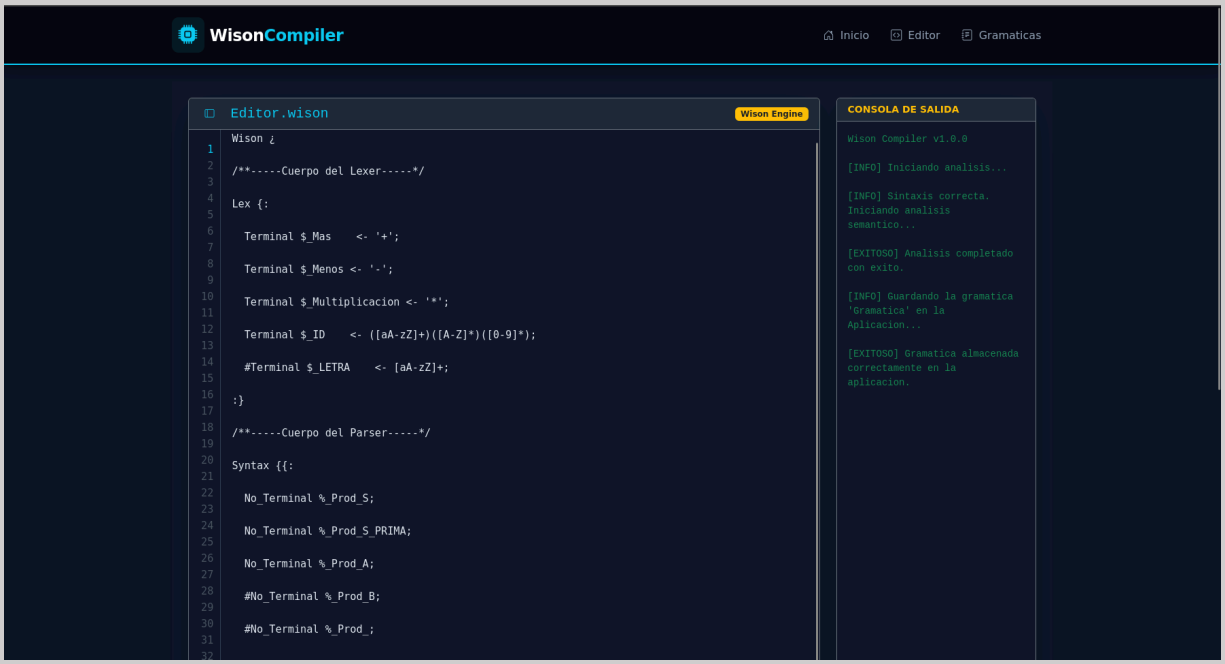


Procesar gramática: Esto le permite al usuario poder procesar la gramática que se ha escrito y poderla guardar en la aplicación para poder probarla posteriormente.

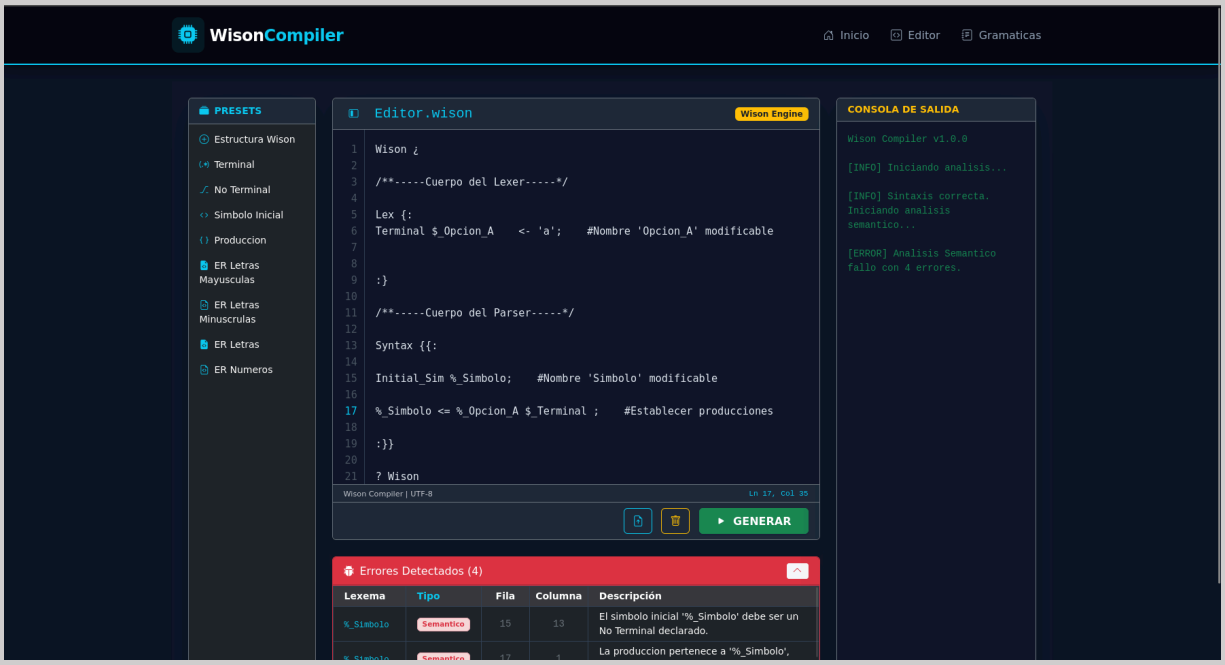




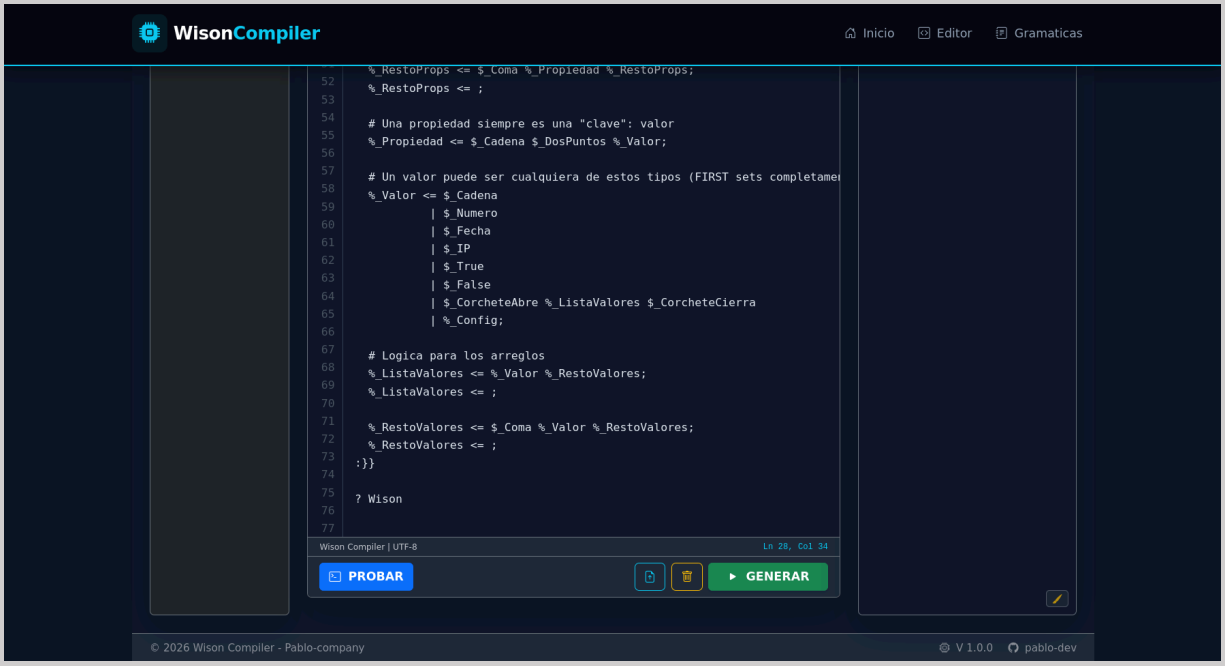
Log informativo: Esta es una consola que permite mostrar todos los procesos que se hacen para que el usuario sea consciente de que su gramática esta siendo procesada y que todo sea exitoso.

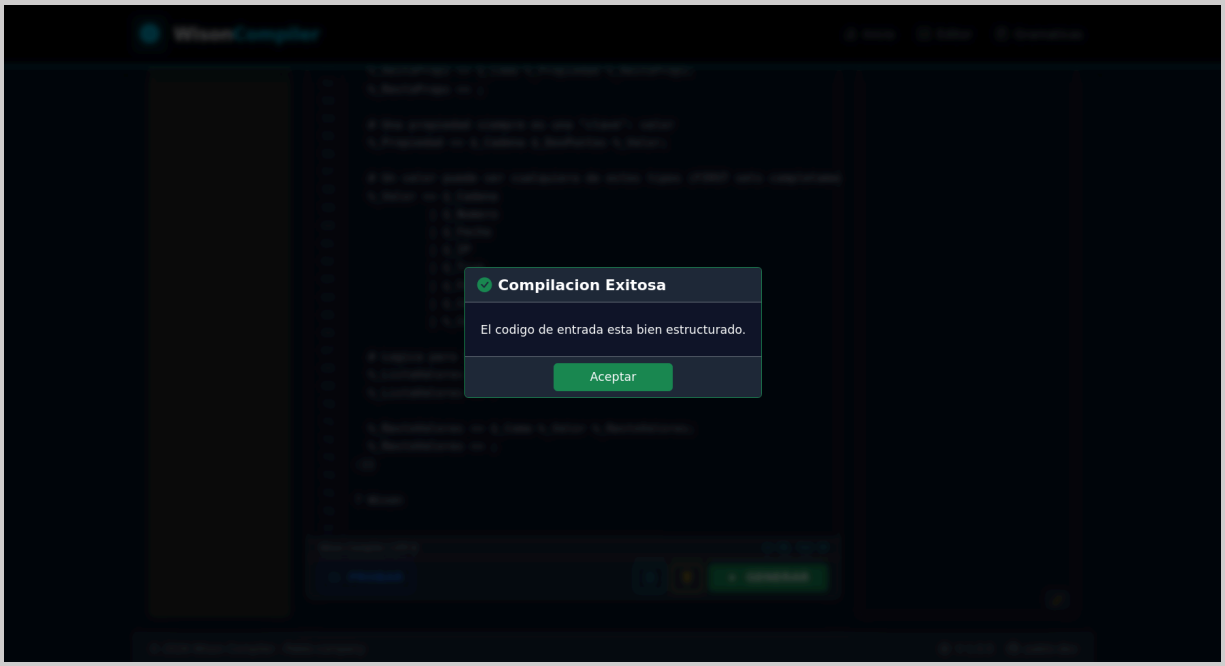


Reconocimiento de errores: Esta es una tabla de errores que indicará el tipo de error que se está registrando en una gramática que se desea generar. Este cuadro sólo aparecerá si hay errores. Caso contrario no aparecerá nada.



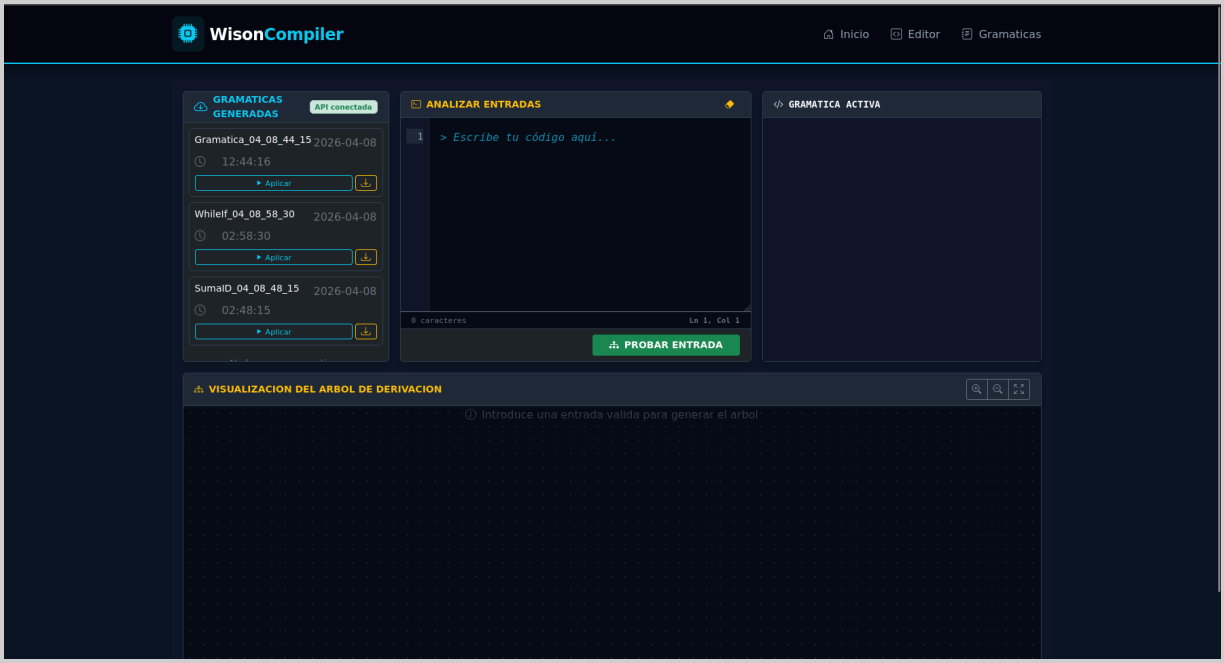
Probar gramática: Esto le permite al usuario poder verificar de forma rápida si su gramática ha sido aceptada o no. Como un paso extra de prueba para poder estar seguros de que se está realizando bien el procedimiento de armar la gramática.





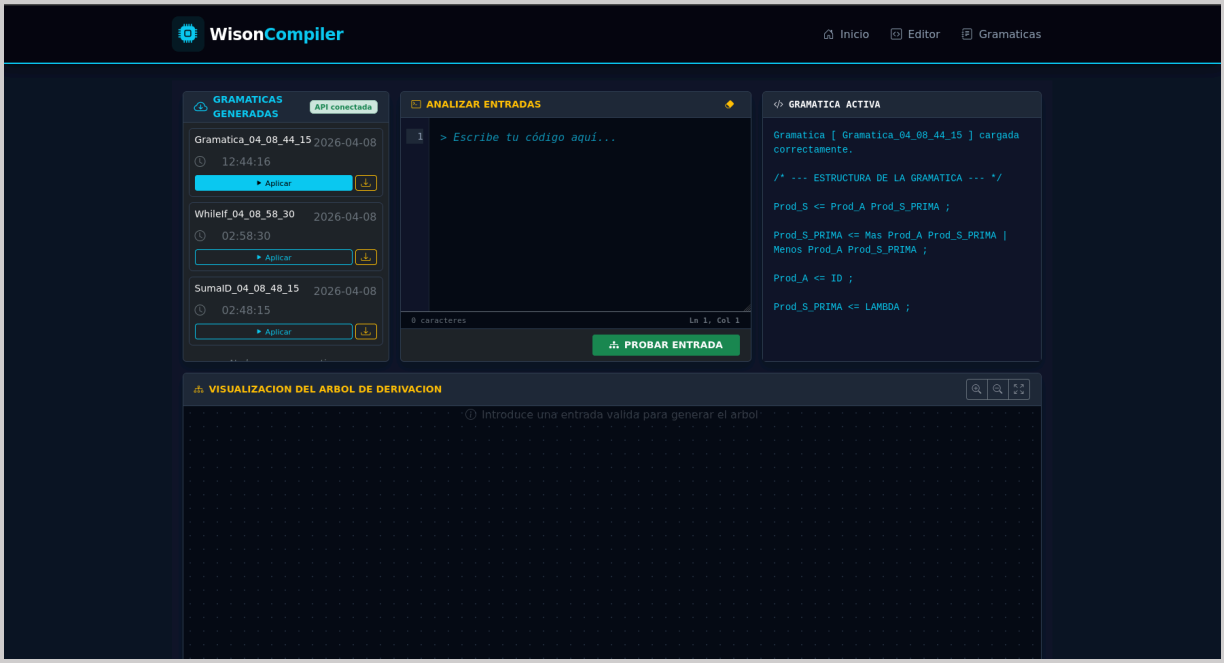
VENTANA DE GRAMÁTICAS

Esta es una sección donde se pueden probar las gramáticas generadas desde la aplicación web. Ya que todo lo que tu generas tiene persistencia en la aplicación puedes seleccionar cualquiera de las gramáticas que alguien haya generado en nuestra web permitiendo de esta forma probar todas las diferentes gramáticas subidas.



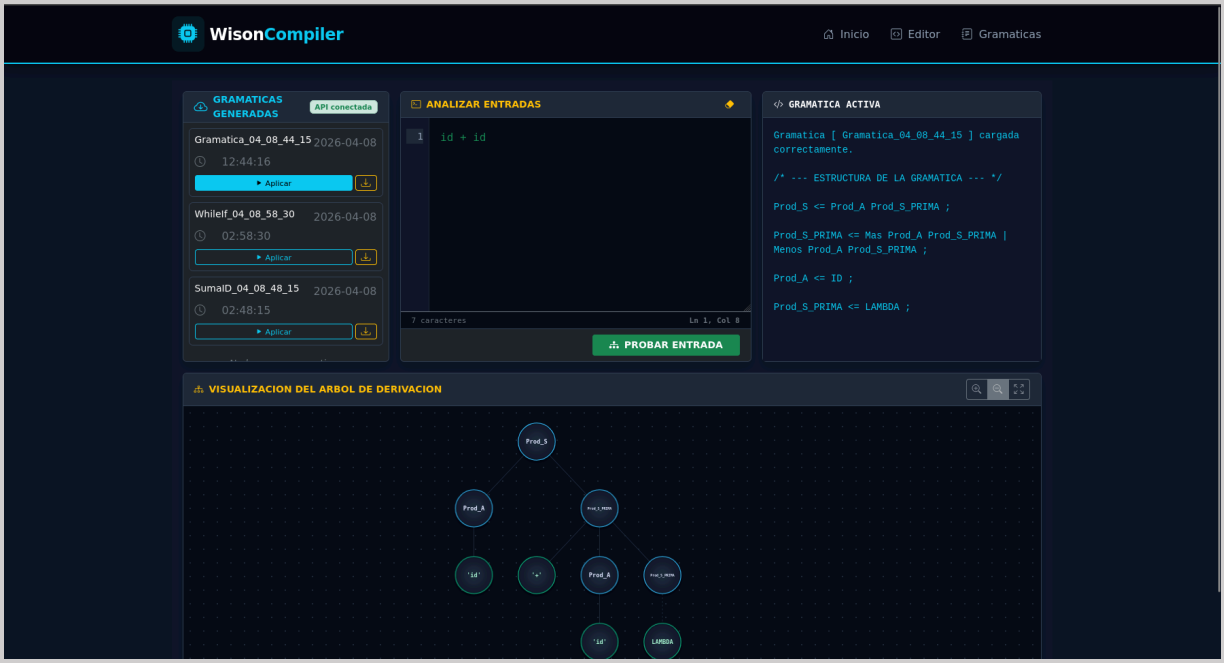
Opciones al cargar una gramática:

Para poder probar una gramática el usuario tiene que presionar en el botón de “aplicar” cuya funcionalidad será cargar temporalmente la gramática para que esta pueda ser utilizada mostrando en la consola del lado derecho las producciones que se tienen de la gramática

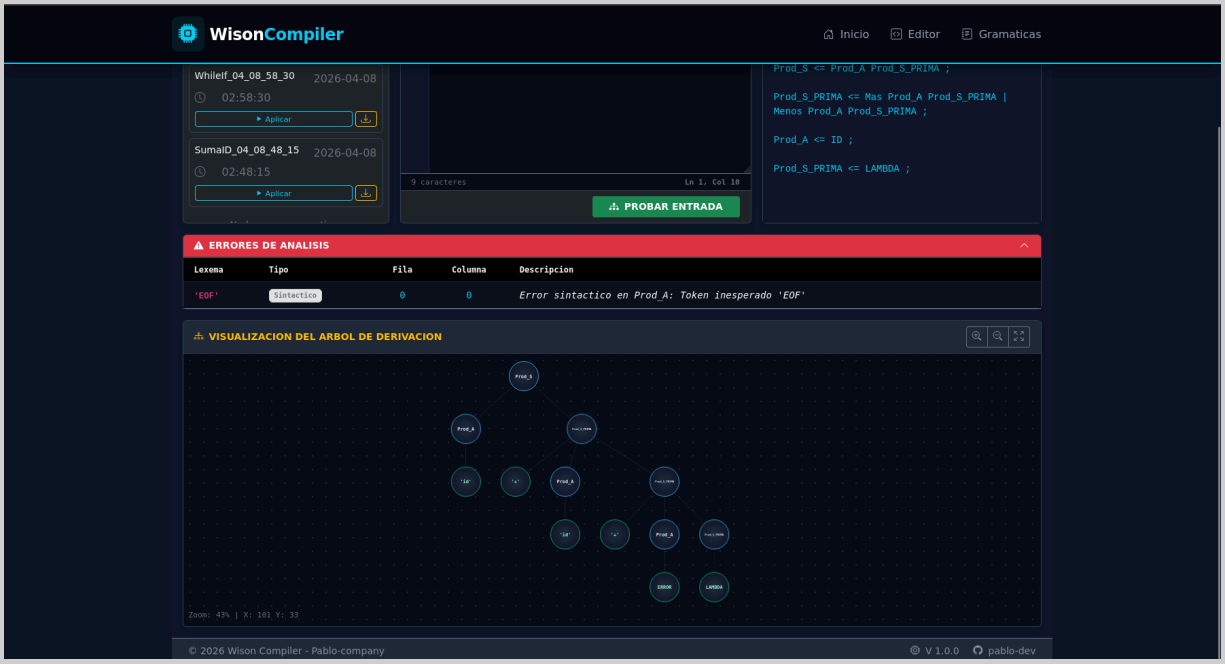


Log de entrada para probar las gramáticas:

Esta es una consola donde el usuario podrá ingresar cadenas de texto que serán analizadas por la propia gramática generada. Se podrá ver el procedimiento que se tiene hasta llegar a un error. Cosa que permite generar la visualización correcta de cómo es que funciona el analizador sintáctico descendente LL(1).



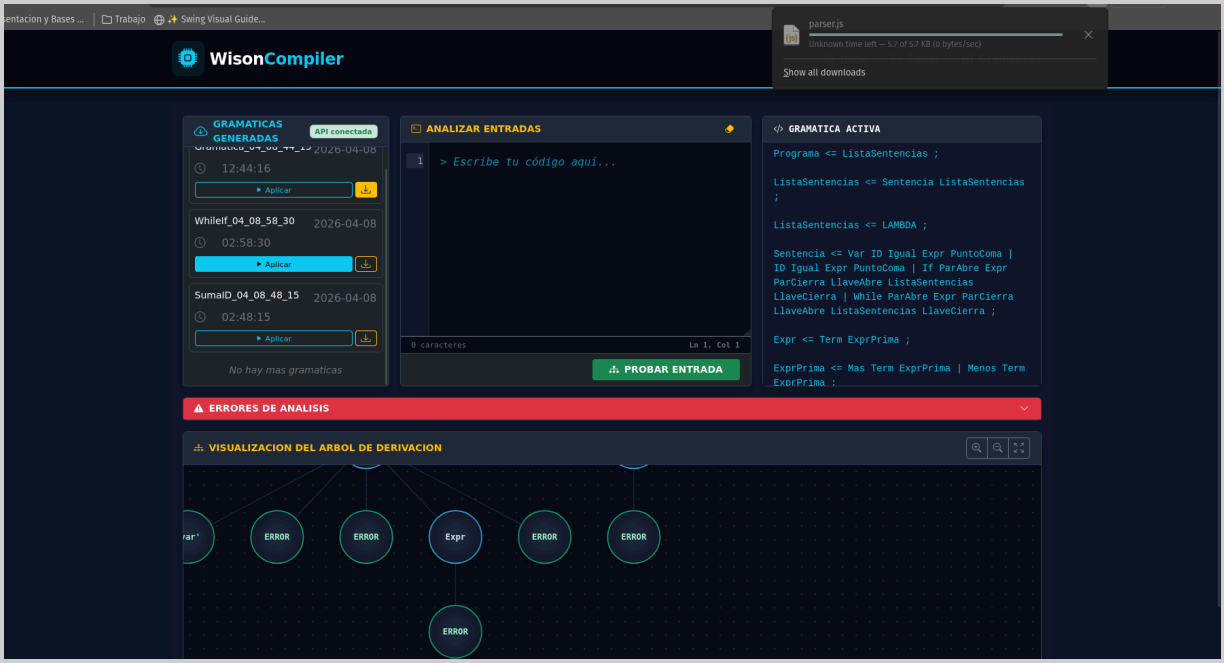
Este árbol de derivaciones cuenta también con la representación de errores. Así que mostrará en todo momento si en alguna cadena de texto no se acepta cierta entrada se generará la transición al estado de error del **parser LL(1)**.



También se cuenta con el reporte de errores propios y también los **expected tokens, mas que todo los terminales que se esperaban como cadena de entrada de esta forma permitiendo que el usuario pueda analizar las cadenas de entrada con las que el parser fue programado.**

Descargar gramática:

Esta es una forma de poder ver cómo es que se generan estos analizadores sintacticos descendentes LL(1). Se genera la clase que permite reconocer las posibles transiciones del LL(1).



DATOS IMPORTANTES SOBRE WISON

Uso de caracteres escapados:

Es importante destacar que el lenguaje **wison** tiene de predeterminado escapar todo término que venga dentro de comillas. por lo tanto si alguien define cierto token como carácter escapado. Este escapara doblemente entonces como tal siempre es importante tener en cuenta que cualquier carácter especial dentro de **wison**. Será automáticamente escapado. Por lo tanto siempre tomarlo en cuenta al utilizar el lenguaje.

- **Forma incorrecta:**
  - **$_Terminal <- ‘\”’;**
- **Forma incorrecta:**
  - **$_Terminal <- ‘”’;**

Uso de macros:

Debido a que **wison** implemente un AFD que sigue la precedencia de expresiones regulares es importante que el usuario sepa que si va a definir una expresión regular antes de otra producción esta se opacara por la que está más arriba y por ende los tokens definidos se harán mal el reconocimiento y por ende la cadena de producciones fallara por la mala tokenización.

- **Forma incorrecta:**
  - **Terminal $_Digito <- [0-9];**
  - **Terminal $_Letra <- [aA-zZ];**
  - **Terminal $_Hex <- ('#') (($_Letra*) ([0-9]*));**

- **Forma incorrecta:**
  - **Terminal $_Letra <- [aA-zZ];**
  - **Terminal $_Hex <- ('#') (($_Letra*) ([0-9]*));**
  - **Terminal $_Digito <- [0-9];**